AI Image Generator

Appendix

Version 2.3
23 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Images, video and music on one GPU: Z-Image Turbo, SDXL fine-tunes, Qwen-Image, FLUX.1, LoRAs, LTX-Video, Wan 2.1, MusicGen and Real-ESRGAN, served locally with Gradio in the Frazile AI studio UI, with per-image GPU telemetry recorded in SQLite and an Admin panel that manages models, LoRAs and downloads and shows live system statistics.

Appendix

A. Project file tree

```
ai-image-gen/
  app.py                  web application: Studio and History tabs, generation
  admin_ui.py             Admin panel: models, LoRAs, add, downloads, system, settings, logs
  registry.py             models/registry.json: settings, model overrides, LoRA metadata
  downloads.py            source inspection and background downloads
  ui_theme.py             colours, Gradio theme, CSS
  ui_common.py            shared UI helpers (choices, header, cards, tiles)
  applog.py               in-memory log for the Logs tab
  media.py                video and music engine: profiles, MediaManager, GPU coordination
  media_ui.py             Video and Music tabs
  video_io.py             ffmpeg / WAV helpers: MP4 encoding, thumbnails, muxing, waveforms
  models.py               model registry, ModelManager, LoRAs, compel
  prompt_parser.py        prompt -> settings
  sysinfo.py              system telemetry for the Admin panel
  upscaler.py             Real-ESRGAN tiled upscaling
  gpu_info.py             GPU facts and live telemetry
  db.py                   SQLite store and CLI viewer
  generate.py             command-line generator
  requirements.txt
  run-app.bat
  README.md
  Dockerfile, docker-compose.yml, .dockerignore
  generations.db          created on first generation
  models/
    RealESRGAN_x4plus.pth
    registry.json         Admin registry (created on first edit)
    cache/                bf16 / NF4 copies of converted models (auto-built)
    checkpoints/          SDXL .safetensors checkpoints (Admin › Add or by hand)
    loras/                LoRA files (Admin › Add or by hand)
  outputs/                generated PNG files
    .thumbs/              History thumbnails
  docs/                   this documentation set as PDFs
  docs-src/
    build_pdf.py          source of the documentation
    images/               figures
  .venv/                  virtual environment (not for backup)
```

B. requirements.txt

```
# Install PyTorch with CUDA first (RTX 3090 / any recent NVIDIA card):
#   pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
# Then:
#   pip install -r requirements.txt
diffusers>=0.31
transformers>=4.45
accelerate>=1.0
safetensors
huggingface_hub
gradio>=5.0
pandas
psutil>=5.9           # Admin panel: CPU, memory, disk and network stats
spandrel>=0.4          # Real-ESRGAN upscaler
peft>=0.13             # LoRA loading
compel>=2.0            # prompt weighting (word:1.3) and long prompts for SDXL models
sentencepiece          # T5 tokenizer for FLUX
protobuf               # T5 tokenizer for FLUX
bitsandbytes>=0.45     # 4-bit FLUX on 24 GB cards
requests               # Admin panel downloads (Civitai, direct URLs)
imageio                # video export (Video tab)
imageio-ffmpeg         # bundled ffmpeg binary: MP4 encoding, thumbnails, soundtrack muxing
soundfile              # WAV export (Music tab)
```

C. Command cheat sheet

All commands run from the project folder in PowerShell with `.\.venv\Scripts\python.exe`.

Task	Command
Start the app	app.py
Generate a category in batch through the app	batch.py --category Nature --count 50 (--prompts file.txt, --model, --seed-start, --list)
Categories and stored prompts	db.py categories · db.py prompts Nature
List models	generate.py --list-models
Generate from CLI	generate.py "prompt" --model juggernaut --seed 42 --count 2
Generate with a LoRA	generate.py "prompt" --model dreamshaper --lora name:0.8
Upscale a file	upscaler.py outputs\x.png 3840
List history / one record / statistics	db.py | db.py 4 | db.py stats
Parser self-test	prompt_parser.py
Model registry status	models.py
Print the Admin registry	registry.py
Inspect a model or LoRA source	downloads.py owner/repo
Start without loading a model	set FRAZILE_STARTUP_MODEL=none, then app.py
Fetch the starter LoRAs	models.py fetch-loras
GPU telemetry dump	gpu_info.py
Full system telemetry as JSON	sysinfo.py
Log in to Hugging Face (for FLUX)	.\.venv\Scripts\huggingface-cli login
Rebuild the PDFs	docs-src\build_pdf.py
Run in Docker	docker compose up --build
Check CUDA	-c "import torch; print(torch.cuda.is_available())"

D. Programmatic access to the running app

```
from gradio_client import Client
c = Client("http://127.0.0.1:7860")
images, report, records, sel, bar, hint, loaded, empty = c.predict(
    "a lighthouse, 16:9, seed 42", "", "juggernaut", [], api_name="/generate")
table, thumbs, stats, models = c.predict("lighthouse", 50, "", False, api_name="/refresh_history"
)
tiles, gpu, cpu, mem, net, power, soft = c.predict(False, api_name="/system_refresh")
tiles, activity, per_model = c.predict(api_name="/overview_refresh")
info, html, *form = c.predict("ostris/crayon_style_lora_sdxl", api_name="/check_source")
log = c.predict(200, "LoRA", "all", api_name="/logs_text")
```

E. Licences

Component	Licence
Stable Diffusion XL base 1.0	CreativeML Open RAIL++-M
Juggernaut XL v9, RealVisXL V5.0, DreamShaper XL v2 Turbo	CreativeML Open RAIL++-M
Playground v2.5	Playground Community License
Animagine XL 3.1	Fair AI Public License 1.0-SD
Z-Image Turbo, Qwen-Image, Qwen-Image-Lightning, FLUX.1 schnell	Apache 2.0
Real-ESRGAN weights and code	BSD 3-Clause

Component	Licence
diffusers, transformers, accelerate, peft, Gradio	Apache 2.0
compel, spandrel, bitsandbytes	MIT
psutil, PyTorch	BSD-style
Project source code	Owned by Frazile AI

Each model's licence is linked from its Hugging Face page. Check the use restrictions before commercial use.